Reinforcement Learning

Clase 11

Ignacio Brevis

ignaciobrve@unisabana.edu.co

Universidad de La Sabana

28 octubre 2026



Repaso: Value function methods

- Transición del caso tabular a funciones

$$v_\pi(s) \quad \leftarrow \quad v(s;w)$$

- Aproximaciones lineales

$$v(s;w) = \sum_{i=1}^{d} w_i \phi_i(s)$$

- Aproximaciones no lineales

$$v(s;w) := nn(\xi(s);w)$$

Si $\rightarrow (f,c)$ Si $\rightarrow$ $\begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$ $S \sim d$.

- Funciones objetivo

$$J(w) = \frac{1}{2}\mathbb{E}\left[\left(v_\pi(S) - \hat{v}(S;w)\right)^2\right].$$

- Algoritmo

$$w_{t+1} = w_t + \alpha_t \Big(v_\pi(s_t) - \hat{v}(s_t;w_t)\Big) \nabla_w \hat{v}(s_t;w_t)$$

TD

Repaso: Value function methods

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1}; w_t) - \hat{v}(s_t; w_t) \Big) \nabla_w \hat{v}(s_t; w_t)$$

TD target

Cuando consideramos una aproximación lineal

$$\begin{aligned} \hat{v}(s; w) &= w_1 \phi_1(s) + w_2 \phi_2(s) + \cdots + w_d \phi_d(s) \\ &= w^\top \phi(s) = \phi(s)^\top w \end{aligned}$$

tenemos que:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \phi(s_{t+1})^\top w_t - \phi(s_t)^\top w_t \Big) \phi(s_t)$$

Obs: El caso lineal es más simple de estudiar y se conocen bien sus propiedades!

See Section 8.2, Zhao and Sections 9.4-9.5, Sutton & Barto.

Repaso: Value function methods (Semi-gradient methods)

GD

$w_{t+1} = w_t - \alpha \nabla J(w_t)$

Adam

$w_{t+1} = w_t - \alpha [\quad]$

$$J(w) = \frac{1}{2}\mathbb{E}\left[\left(v_\pi(s) - \hat{v}(s;w)\right)^2\right]$$

$$w_{t+1} = w_t + \alpha_t \Big(v_\pi(s_t) - \hat{v}(s_t;w_t)\Big)\nabla_w \hat{v}(s_t;w_t)$$

GD

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{v}(s_{t+1};w_t) - \hat{v}(s_t;w_t)\Big)\nabla_w \hat{v}(s_t;w_t)$$

semi-gradient methods

$$J(w_t) = \mathbb{E}\left[\left(R_{t+1} + \gamma \hat{v}(S_{t+1};w_t) - \hat{v}(S_t;w_t)\right)^2\right]$$

Repaso: Value function methods (Objective functions)

Nota sobre las funciones objetivo

- **True value error**:

$$J_{\mathrm{E}}(w) = \mathbb{E}\Big[(v_\pi(S) - \hat{v}(S;w))^2\Big] = \|v_\pi - \hat{v}(w)\|_D^2,$$

donde $\|x\|_D^2 = x^\top D x$ y $D = \begin{bmatrix} \ddots & & \\ & d_\pi(s) & \\ & & \ddots \end{bmatrix}$.

- **Bellman error**:

$$J_{\mathrm{BE}}(w) = \|\hat{v}(w) - \underbrace{(r + \gamma P_\pi \hat{v}(w))}_{T_\pi(\hat{v}(w))}\|_D^2$$

- **Projected Bellman error**:

$$J_{\mathrm{PBE}}(w) = \|\hat{v}(w) - M T_\pi(\hat{v}(w))\|_D^2$$

Repaso: Learning action values (SARSA)

SARSA nos permite estimar el **valor de la acción** por medio del algoritmo de TD:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}; w_t) - \hat{q}(s_t, a_t; w_t) \Big) \nabla_w \hat{q}(s_t, a_t; w_t)$$

TD target

donde $t = 0, 1, 2, \ldots$

- En cada paso t requiere $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}$
- Calcula el valor de la acción para una política dada aproximando la solución de la siguiente ecuación de Bellman:

$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

- Junto a un paso de mejora de política **puede calcular políticas óptimas**

Repaso: Learning action values (SARSA)

Algorithm 8.2: Sarsa with function approximation

Initialization: Initial parameter w_0. Initial policy π_0. $\alpha_t = \alpha > 0$ for all t. $\epsilon \in (0, 1)$.
Goal: Learn an optimal policy that can lead the agent to the target state from an initial state s_0.
For each episode, do
  Generate a_0 at s_0 following $\pi_0(s_0)$
  If s_t ($t = 0, 1, 2, \dots$) is not the target state, do
    Collect the experience sample $(r_{t+1}, s_{t+1}, a_{t+1})$ given (s_t, a_t): generate r_{t+1}, s_{t+1} by interacting with the environment; generate a_{t+1} following $\pi_t(s_{t+1})$.
    Update q-value:
    $w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$ GD apuesto
    Update policy:
    $\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|}(|\mathcal{A}(s_t)| - 1)$ if $a = \arg\max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1})$
    $\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|}$ otherwise
    $s_t \leftarrow s_{t+1}$, $a_t \leftarrow a_{t+1}$

Page 180, Zhao, Mathematical foundations of reinforcement learning

Repaso: Learning action values (Q-learning)

Q-learning nos permite estimar la **acciones óptimas** por medio del algoritmo de TD:

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \max_{s' \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a'; w_t) - \hat{q}(s_t, a_t; w_t) \Big) \nabla_w \hat{q}(s_t, a_t; w_t)$$

donde $t = 0, 1, 2, \ldots$

- En cada paso t requiere $\{(s_t, a_t, r_{t+1}, s_{t+1})\}$
- Calcula el valor de acciones óptimas aproximando la solución de la siguiente ecuación optimal de Bellman:

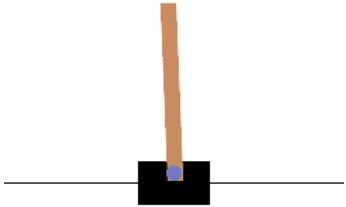
$$q(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') | S_t = s, A_t = a]$$

- No necesita de un paso de mejora de política para calcular políticas óptimas

Repaso: Learning action values (Example)

En `https://github.com/ibrevis/RL_unisabana_2026_2`,

- Ir a `/Lesson10/`
- Abrir `01_CartPole_intro.ipynb` para una introducción guiada al entorno
- Ir a `Example_03_deep_SARSA_CartPole.ipynb` para entrenar la política.
- Comprueba tues resultados con `run_deep_sarsa_cartpole_policy.py`.
- Estudiar qué sucede si modificamos el "semi-gradient"



Repaso: Learning action values (Example)

```python
def sarsa_update(state, action, reward, next_state, next_action, terminated):
    """One semi-gradient SARSA step on the transition (s, a, r, s', a')."""
    state_tensor = torch.as_tensor(state, dtype=torch.float32).unsqueeze(0)
    next_state_tensor = torch.as_tensor(next_state, dtype=torch.float32).unsqueeze(0)

    # q_hat(s_t, a_t; w): this is the term we differentiate
    q_value = q_net(state_tensor)[0, action]

    # TD target: treated as a constant (no gradient)
    with torch.no_grad():
        next_q_value = q_net(next_state_tensor)[0, next_action]
        target = reward + GAMMA * next_q_value * (1.0 - terminated)

    loss = loss_fn(q_value, target)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()
```

$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}; w_t) - \hat{q}(s_t, a_t; w_t) \Big) \nabla_w \hat{q}(s_t, a_t; w_t)$$

Value function methods: Deep Q-learning

- Primer trabajo en utilizar Deep NNs en RL de forma exitosa.
- Presentado en NIPS Deep Learning Workshop, 2013.
- En el 2015 fue publicado en Nature bajo el nombre “Human-level control through deep reinforcement learning” (vol. 518)
- Utilizando CNN logró jugar juegos de Atari (e incluso sobrepasar a humanos expertos)

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih **Koray Kavukcuoglu** **David Silver** **Alex Graves** **Ioannis Antonoglou**
Daan Wierstra **Martin Riedmiller**
DeepMind Technologies
{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.



Figure 1: Screen shots from five Atari 2600 Games: (*Left-to-right*) Pong, Breakout, Space Invaders, Seaquest, Beam Rider

Image from Playing Atari with Deep Reinforcement Learning, 2013.

Value function methods: Deep Q-learning

Hasta ahora hemos revisado expresiones de la forma:

$$\boxed{w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \max_{s' \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a'; w_t) - \hat{q}(s_t, a_t; w_t) \Big) \nabla_w \hat{q}(s_t, a_t; w_t)}$$

Que implican operaciones "low-level" y limitan la expresión a un método en particular (gradient descent).

De forma más general, deep Q-learning busca minimizar la función de costo:

$$J(w) = \frac{1}{2} \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w) - \hat{q}(S, A; w) \right)^2 \right],$$

donde (S, A, R, S') son variables aleatorias.

Value function methods: Deep Q-learning

Deep Q-learning busca minimizar la función de costo:

$$J(w) = \frac{1}{2}\mathbb{E}\left[\left(\underbrace{R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w)}_{\text{TD target}} - \hat{q}(S, A; w)\right)^2\right],$$

Este funcional busca minimizar el **Bellman optimality error**

$$\text{BOE}: \boxed{q(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') | S_t = s, A_t = a]}$$

Esto es, $R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w) - \hat{q}(S, A; w)$ debe ser cero en bajo el valor esperado.

Para minimizar el funcional, volvamos al caso de gradient descent!

$$J(w) = \frac{1}{2}\mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w) - \hat{q}(S, A; w)\right)^2\right],$$

y_T

$\nabla_w J(w)$

Obs: El parametro w no solo aparece en $\hat{q}(S, A; w)$, sino que además en

$$y_T := R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; w)$$

Esto nos genera un problema a la hora de calcular el gradiente $\nabla J(w)$!

Value function methods: Deep Q-learning

- Primero definimos dos redes neuronales

main network : $\hat{q}(s,a;w)$

target network : $\hat{q}(s,a;\eta)$

- La función objetivo queda de la forma:

$$J(w,\eta) = \frac{1}{2}\mathbb{E}\left[\left(R + \gamma \max_{a'} \hat{q}(S',a';\eta) - \hat{q}(S,A;w)\right)^2\right]$$

$$\nabla_w J(w,\eta)$$

Value function methods: Deep Q-learning

Dado el funcional:

$$\boxed{J(w,\eta) = \frac{1}{2}\mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S',a;\eta) - \hat{q}(S,A;w)\right)^2\right]}$$

podemos calcular el gradiente fácilmente cuando η está fijo:

$$\nabla_w J(w,\eta) = \mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S',a;\eta) - \hat{q}(S,A;w)\right) \nabla_w \hat{q}(S,A;w)\right]$$

- Experience replay
- Neural network structure

Experience replay

- Obtenemos experiencias a partir de la behavior policy.

$$(s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots)$$

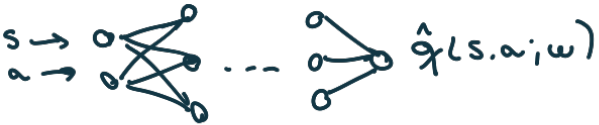
- En vez de utilizar las muestras/experiencias en el orden en que fueron obtenidas, las almacenamos en un conjunto llamado **replay buffer**

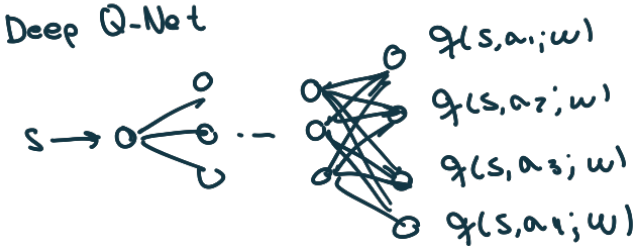
$$\mathcal{B} := \{(s, a, r, s')\}$$

- Para entrenar la red neuronal, seleccionamos aleatoriamente elementos de $\mathcal{B}$ para generar un mini-batch.

- Utilizar las experiencias de esta forma se conoce como **experience replay**

Neural network structure





Value function methods: Q-learning

Deep Q-learning nos permite estimar la **acciones óptimas** por medio del algoritmo de TD:



$$w_{t+1} = w_t + \alpha_t \Big(r_{t+1} + \gamma \max_{s' \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a'; \eta) - \hat{q}(s_t, a_t; w_t) \Big) \nabla_w \hat{q}(s_t, a_t; w_t)$$

donde $t = 0, 1, 2, \ldots$

- Actualiza η cada cierta cantidad de pasos. Entonces $\eta = w$
- Para cada t obtiene $\{(s_t, a_t, r_{t+1}, s_{t+1})\}$ desde un replay buffer (**experience replay**)
- La estructura de la red neuronal toma s como input, y devuelve

$$\begin{bmatrix} \hat{q}(s, a_1) \\ \hat{q}(s, a_2) \\ \vdots \\ \hat{q}(s, a_m) \end{bmatrix}$$

Value function methods: Q-learning

Algorithm 8.3: Deep Q-learning (off-policy version)

Initialization: A main network and a target network with the same initial parameter.
Goal: Learn an optimal target network to approximate the *optimal* action values from the experience samples generated by a given behavior policy π_b.

Store the experience samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$
  For each iteration, do
    Uniformly draw a mini-batch of samples from $\mathcal{B}$
    For each sample (s, a, r, s'), calculate the target value as $y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$, where w_T is the parameter of the target network
    Update the main network to minimize $(y_T - \hat{q}(s, a, w))^2$ using the mini-batch of samples
  Set $w_T = w$ every C iterations

$\eta := w_T$

(ver versión on-policy en página 181)

Page 184, Zhao, Mathematical foundations of reinforcement learning

The Deadly Triad

Cautionary tale: Combinar estos tres elementos puede producir inestabilidades en los algoritmos:

- **Function approximation**: Aproximación lineal y no lineal.
- **Bootstrapping**: Como los algoritmos de temporal difference (TD) learning.
- **Off-policy training**: Entrenamiento con una política distinta de la producida por la política objetivo.

Ejemplo: El contraejemplo de Baird (ver Sección 11.2 de Sutton & Barto).

Obs: "Q-learning has never been found to diverge in this case, but there has been no theoretical analysis" (2018).

Sección 11.3, Sutton & Barto, 2018

The Deadly Triad

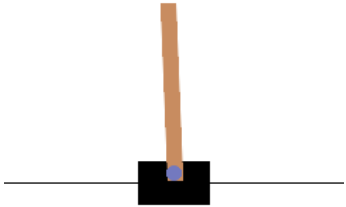
- Jianqing Fan, Zhaoran Wang, Yuchen Xie, Zhuoran Yang, **A Theoretical Analysis of Deep Q-Learning**, *Proceedings of Machine Learning Research*, Vol 120, 1–4, 2020.
- Zhikang T. Wang, Masahito Ueda, **Convergent and Efficient Deep Q Learning Algorithm**, Published as a conference paper at ICLR 2022.
- Shuai Zhang et al., **On the Convergence and Sample Complexity Analysis of Deep Q-Networks with ϵ-Greedy Exploration**, 37th Conference on Neural Information Processing Systems (NeurIPS 2023).

Deep Q-learning: Examples

See `https://gymnasium.farama.org/environments/classic_control/cart_pole/` for a guided introduction to the environment

In `https://github.com/ibrevis/RL_unisabana_2026_2`

- go to `/Lesson11/`
- Go `01_DQN_CartPole.ipynb` and tune the hyperparameters to traing the policy.
- Check your results with `run_dqn_cartpole_policy.py`.



Deep Q-learning: Examples

See `https://gymnasium.farama.org/environments/box2d/lunar_lander/` for a guided introduction to the environment

In `https://github.com/ibrevis/RL_unisabana_2026_2`

- go to `/Lesson11/`
- Go `02_DQN_LunarLander.ipynb` and complete the function `train_step()` to traing the policy.
- Check your results with `run_dqn_lunarlander_policy.py`.

